

DAT62-4

Computer Vision

An applied course

Building computer vision systems from image preprocessing through model training to production deployment.

Albert School — 22 hours

Contents

1	Image representation	2
2	Data augmentation	2
3	Building and training a CNN from scratch	2
4	Pretrained models	3
5	Evaluation under constraints	3
6	Object detection	3
7	Computer vision task taxonomy	3
8	Production failure modes	4
9	Model interpretation with Grad-CAM	4
10	Complete computer vision project	4

1 Image representation

This course assumes the implementation skills of DAT62-2 — building and training networks in PyTorch, CNN architecture (convolutional layers, pooling, receptive fields), and transfer learning with pretrained models such as ResNet and EfficientNet. It applies them to computer vision and does not re-teach them.

An image is represented as a **tensor**. A color image is a tensor of shape (channels, height, width): three channels for the red, green, and blue components, each a grid of pixel intensities. A grayscale image has one channel.

- **Resolution** is the height and width in pixels. Higher resolution preserves more detail but increases computation and memory; lower resolution is cheaper but discards fine structure. The resolution chosen affects model performance.
- **Color space** is how color is encoded (e.g. RGB, or grayscale). The choice of color space affects what information the model receives.
- **Normalization** rescales pixel values (typically to zero mean and unit variance per channel, using dataset statistics) so inputs are on a consistent scale. This affects how well and how fast the model trains.

2 Data augmentation

A **data augmentation pipeline** applies random transformations to training images so the model sees varied versions of each example. Each transformation teaches an **invariance** — a change to which the model’s prediction should not change:

- **Random crop**: take a random sub-region of the image. Teaches invariance to position and framing.
- **Flip** (e.g. horizontal): mirror the image. Teaches invariance to left–right orientation.
- **Rotation**: rotate the image by a random angle. Teaches invariance to orientation.
- **Color jitter**: randomly perturb brightness, contrast, and saturation. Teaches invariance to lighting and color conditions.

Each transformation is justified by the invariance it teaches: it should be applied only when the label genuinely does not change under it (for example, a horizontal flip is inappropriate when left–right orientation carries meaning).

3 Building and training a CNN from scratch

A **convolutional neural network** (CNN) is built and trained from scratch on an image classification task. Its layers play distinct roles:

- **Convolutional layers** slide kernels over the input to produce feature maps, detecting local patterns wherever they occur.
- **Pooling layers** reduce the spatial size of feature maps, lowering computation and adding tolerance to small shifts.
- **Fully connected layers** at the end map the extracted features to class scores.

The convolutional and pooling layers extract features from local patterns up to larger structures; the fully connected layers turn those features into a prediction over the classes.

4 Pretrained models

Pretrained models such as **ResNet** and **EfficientNet** have already been trained on large image collections and provide layers that extract general visual features. There are two ways to reuse them:

- **Feature extraction:** freeze the pretrained layers and train only a new classifier on top. The pretrained weights are unchanged. Appropriate when the new dataset is small or similar to the original, and when training must be cheap.
- **Fine-tuning:** continue training some or all of the pretrained layers on the new dataset, usually with a small learning rate. Appropriate when the new dataset is larger or differs more from the original.

The two approaches are compared on a task, and the choice is justified by the size of the available data and its similarity to the pretraining data.

5 Evaluation under constraints

Choosing an evaluation metric depends on the problem's constraints:

- **Class imbalance:** when some classes are much rarer than others, accuracy is misleading because predicting the majority class scores well. Metrics such as precision, recall, and F1 — reported per class — reflect performance on the rare classes.
- **Business cost asymmetry:** when different errors carry different costs (a missed detection may cost far more than a false alarm), the metric must weight errors accordingly, favoring for example recall over precision when misses are costly.

The metric is selected to match the class distribution and the relative cost of the errors.

6 Object detection

Object detection locates and classifies objects in an image, outputting a **bounding box** and a class for each detected object. It is implemented with a **pretrained model** such as **YOLO**.

- A **bounding box** is a rectangle (position and size) around a detected object, with a class label and a confidence score.
- **Intersection over Union (IoU)** measures the overlap between two boxes: the area of their intersection divided by the area of their union. It is used to decide whether a predicted box matches a ground-truth box.
- **Non-maximum suppression (NMS)** removes duplicate detections of the same object: among overlapping boxes (high IoU) for a class, it keeps the one with the highest confidence and discards the rest.

7 Computer vision task taxonomy

Three computer vision tasks, each with a concrete business use case:

- **Classification:** assign one label to a whole image. Use case: sorting product photos into categories for an online catalog.
- **Detection:** locate and classify each object with a bounding box. Use case: counting items on a retail shelf for stock monitoring.
- **Segmentation:** label each pixel with the class of the object it belongs to. Use case: outlining a tumor region in a medical scan.

Classification answers *what is in the image*, detection *what is where*, and segmentation *which pixels belong to what*.

8 Production failure modes

Real computer vision systems are fragile in production. Three common failure modes, each with a mitigation:

- **Distribution shift:** the images seen in production differ from the training data (different lighting, cameras, or backgrounds), so performance drops. Diagnosed by comparing production inputs and accuracy to training conditions. Mitigated by collecting and retraining on representative data, and by monitoring inputs.
- **Label noise:** some training labels are wrong, which limits and misleads learning. Diagnosed by auditing labels, especially on misclassified examples. Mitigated by cleaning or relabeling the data and improving labeling guidelines.
- **Long-tail class imbalance:** a few classes have many examples while many classes have very few, so rare classes are predicted poorly. Diagnosed by inspecting the class distribution and per-class metrics. Mitigated by resampling, class weighting, or targeted data collection for rare classes.

9 Model interpretation with Grad-CAM

Grad-CAM visualizes which regions of an image drive a model’s prediction. It produces a heatmap highlighting the parts of the image most responsible for the predicted class, overlaid on the original image.

It is used to check that the model attends to relevant regions (the object itself rather than background or spurious cues) and to **communicate to a non-technical stakeholder** — the heatmap shows, in plain visual terms, why the model made its prediction.

10 Complete computer vision project

The course culminates in a complete project. The student must:

1. **define the task** (classification, detection, or segmentation) for a concrete use case;
2. **build and augment the dataset**, applying a justified augmentation pipeline;
3. **train and fine-tune** a model, using a pretrained model where appropriate;
4. **evaluate rigorously**, selecting metrics suited to the class distribution and error costs;
5. **present the results with a model card** — a short document recording the model’s purpose, data, architecture, training setup, evaluation results, and limitations.